

Dokumentacja Architektury Systemu Wdrażanie Minimalnej Architektury Przetwarzania Danych

Michał Proc 198407, Maciej Skibiński 198247

24.06.2026

Wstęp

Zgodnie z wymogami projektu, wdrożony system monitorowania jakości danych meteorologicznych realizuje sześćoetapową, zalecaną architekturę przetwarzania danych (*Recommended Minimal Architecture*). Poniższy dokument prezentuje sposób implementacji poszczególnych warstw w autorskim kodzie napisanym w języku Python.

1 REST API Collector

Pobieranie najnowszych obserwacji pogodowych realizowane jest za pomocą dedykowanej funkcji wykorzystującej bibliotekę `requests`. Endpoint odpytywany jest cyklicznie w głównej pętli programu.

```
1 def fetch_api(endpoint: str, params: dict) -> dict:
2     url = f"{API_BASE_URL}{endpoint}"
3     resp = requests.get(url, headers=HEADERS, params=params)
4     resp.raise_for_status()
5     return resp.json()
6
7 # Wywołanie:
8 raw_data = fetch_api("/latest", {"station_id": STATION_ID})
```

2 Zapisywanie danych

Każda pomiar pobrany z interfejsu API jest natychmiast, bez żadnych modyfikacji, trwale zapisywana w strukturze surowej (*Raw Storage*) w formacie JSONL.

```
1 def save_to_jsonl(filename: str, data: dict):
2     with open(filename, 'a', encoding='utf-8') as f:
3         f.write(json.dumps(data) + "\n")
4
5 # Archiwizacja surowego pakietu:
6 save_to_jsonl(f"raw_data_{STATION_ID}.jsonl", raw_data)
```

3 Walidacja Pomiarów

Główna oś programu, w której obiekt klasy `DataQualityValidator` weryfikuje kompletność oraz zakresy wartości. Sprawdzane jest m.in. występowanie wartości typu `Null` oraz odchylenia od limitów fizycznych.

```
1 # Ekstrakcja i walidacja zakresow
2 if not (-50 <= current.temperature <= 50):
3     errors.append(f"Temperature {current.temperature} poza zakresem")
4 if not (0 <= current.humidity <= 100):
5     errors.append(f"Humidity {current.humidity} poza zakresem")
6 if not (980 <= current.pressure <= 1080):
7     errors.append(f"Pressure {current.pressure} poza zakresem")
```

4 Wynik i rekomendacje programu

Na podstawie zidentyfikowanych anomalii aplikacja generuje rekomendacje do poprawy pomiarów (*Cleaning Recommendations*) oraz wyliczając metrykę *Quality Score*.

```
1 # Obliczanie metryki jakosci dla danego rekordu
2 error_count = len(errors)
3 total_params = validator.total_params_per_record
4 quality_score = ((total_params - min(error_count, total_params)) /
5                  total_params) * 100
6
7 # Dobieranie rekomendacji transformacji danych (np. imputacja)
8 if "Brak danych" in err:
9     recs.add("Uzyj 'forward fill' (ffill) w Pandas.")
10 elif "poza zakresem" in err or "Skok" in err:
11     recs.add("Zmien bledna wartosc na Null i uzyj interpolacji liniowej.")
```

5 Analiza i reguły eksperckie

Zaawansowane reguły analityczne analizują dane w szerszym kontekście historycznym (wykorzystując bufor 10 ostatnich pomiarów). Pozwala to na detekcję złożonych błędów, takich jak uszkodzone czujniki oraz naruszenia spójności meteorologicznej.

```
1 # Regula ekspercka: Deszcz nie może występować przy bezchmurnym niebie
2 if current.rain_mm > 5 and current.cloud_cover <= 5:
3     errors.append("Spójność: Deszcz przy braku chmur")
4
5 # Analityka szeregu czasowego: Detekcja zablokowanego czujnika
6 if self._check_stuck('temperature', 10, current.temperature):
7     errors.append("Stuck sensor: Temp (10)")
```

6 Wykrywanie i zapisywanie anomalii

Gdy silnik walidacyjny zidentyfikuje anomalię, system zapisuje taki pomiar w osobnym pliku `quarantine_invalid_GDN_01.jsonl`. Błędny odczyt nie przerywa działania potoku danych.

Program tworzy strukturę słownikową, która agreguje oryginalny, surowy odczyt JSON z interfejsu API, listę oflagowanych naruszeń, wygenerowane systemowo rekomendacje czyszczenia (*cleaning recommendations*) oraz obliczony, obniżony wskaźnik *Quality Score*.

```
1 # Jesli walidator zwroci jakiegokolwiek bledy:
2 if errors:
3     # 1. Logowanie problemow do konsoli operacyjnej
4     for err in errors:
5         print(f"      * BLAD: {err}")
6     for rec in recommendations:
7         print(f"      * REKOMENDACJA: {rec}")
8
9     # 2. Tworzenie wpisu kwarantanny z pelnym kontekstem
10    quarantine_entry = {
11        "raw_record": raw_data,
12        "errors": errors,
13        "cleaning_recommendations": recommendations,
14        "quality_score_percent": quality_score
15    }
16
17    # 3. Trwaly zapis wyizolowanego rekordu do dedykowanego pliku
18    save_to_jsonl(f"quarantine_invalid_{STATION_ID}.jsonl",
19 quarantine_entry)
20 else:
21     print(" -> Status: OK (Brak bledow, rekord czysty)")
```

7 Raport z grupy 10 pomiarów

System archiwizuje ocenę każdego pomiaru do stałego logu raportującego, generując dodatkowo wpisy kumulatywne.

```
1 # Dynamiczne logowanie poprawnego lub blednego rekordu
2 report_entry = {
3     "timestamp": record.timestamp,
4     "status": "blad" if errors else "rekord poprawny",
5     "quality_score_percent": quality_score
6 }
7 save_to_jsonl(f"report_{STATION_ID}.jsonl", report_entry)
8
9 # Separacja blednych paczek (Kwarantanna)
10 if errors:
11     quarantine_entry = {
12         "raw_record": raw_data,
13         "errors": errors,
14         "cleaning_recommendations": recommendations,
15         "quality_score_percent": quality_score
16     }
17     save_to_jsonl(f"quarantine_invalid_{STATION_ID}.jsonl",
18 quarantine_entry)
```